

UNIVERSITY OF THESSALY
Department of Electrical & Computer Engineering
School of Science · Undergraduate Programme

ADVANCED DATA MANAGEMENT

Final Assignment Report

GridSense — Smart Power Grid Analytics & Fault Management

Students: Vaggelis Plytas, Thanasis Kampouris

Student ID: 3674, 3655

Academic Year 2025–2026 · Spring Semester

GitHub: <https://github.com/ThanasisKam/GridSense>

Contents

- Part A — Critical Thinking
- Part B — Implementation Overview
- Part C — Measurement
- Part D — Reflection
- Bibliography
- Appendix A — AI Use Disclosure

Part A — Critical Thinking

A.1 — Why Relational Databases Were Not Enough

a) The merit in the senior engineer's position

The engineer is correct that PostgreSQL with TimescaleDB or native partitioned tables can handle time-series workloads in a disciplined environment. A properly designed hypertable with chunk-based partitioning by time can ingest hundreds of thousands of rows per second on modern NVMe hardware, and TimescaleDB has real production deployments handling dozens of terabytes. The operational argument is also genuine: running one system means one backup strategy, one team skill set, one monitoring stack, and one failure mode. For an organisation like the RPA that already has PostgreSQL expertise, adding four unfamiliar systems creates real hiring, training, and on-call costs. That is not a trivial objection.

b) Where the argument breaks down — with calculation

The stated requirement is 40,000 events/second at normal load, 120,000/s during storm events. A single sensor reading is approximately 80–120 bytes. Taking 100 bytes as a working estimate:

- Sustained write throughput: $40,000 \text{ events/s} \times 100 \text{ bytes} = 4 \text{ MB/s}$
- Storm-event throughput: $120,000 \text{ events/s} \times 100 \text{ bytes} = 12 \text{ MB/s}$

The throughput itself is manageable for a single server. The problem is PostgreSQL's write amplification across four dimensions:

- WAL: Every INSERT writes the full row to the WAL before the heap. At 40,000 rows/s, this means ~8 MB/s to WAL alone — flushed to disk before the transaction can be acknowledged.
- VACUUM: PostgreSQL's MVCC marks deleted rows as dead tuples. At 40,000 inserts/s with a 90-day TTL, dead rows accumulate to ~3.4 billion per window. VACUUM on this scale causes significant I/O contention and periodic latency spikes violating the <50 ms SLA.
- Index maintenance: A B-tree index on (sensor_id, reading_time) must be updated for every insert. At 40,000 inserts/s, index pages become hotspots under concurrent write load.
- Heap fragmentation: PostgreSQL's heap storage is not log-structured. Reads of recent data traverse non-sequential pages as the table grows past available RAM.

Cassandra avoids all four: its LSM-tree write path appends to a memory buffer and commit log only; there is no VACUUM equivalent; and on a 10-node cluster each node handles just 4,000 events/s — well within capacity, scaling linearly with nodes added.

c) When a PostgreSQL-only design would be correct

A PostgreSQL-only design is justified when:

- Write rate stays below ~5,000 events/second sustained (well-tuned PostgreSQL with TimescaleDB handles this without VACUUM contention)

- The dataset is expected to stay under ~1 TB total, where a partitioned hypertable with pg_partman remains manageable
- The organisation has strong PostgreSQL expertise but no budget for Cassandra operational training
- Sensor data must be joined with billing or topology data in ad-hoc queries (Cassandra cannot do cross-partition JOINS)
- The deployment is single-region with no horizontal scaling requirement

In summary: if the RPA had 400 sensors instead of 3.4 million, PostgreSQL would be the right answer.

A.2 — The CAP Theorem Applied

a) The CAP trade-off — precisely

Gilbert and Lynch (2002) state CAP precisely: in an asynchronous network where message delays are unbounded, a distributed system cannot simultaneously guarantee Consistency (every read returns the most recent write or an error), Availability (every request to a non-failed node returns a response), and Partition Tolerance (the system continues operating despite network partitions). Partition tolerance is not optional — a system that stops working during a partition has abandoned both availability and consistency. The real choice is between C and A during a partition event.

Engineer A (ONE/ONE) makes an AP choice: the system remains available during a partition but may return stale data. Engineer B (QUORUM/QUORUM) makes a CP choice: the system preserves consistency but may refuse operations when quorum cannot be reached.

b) Minimum consistency for reads to see most recent writes — calculation

With $RF = 3$ across two data centres, the requirement for strong consistency is $W + R > RF$, meaning $W + R \geq 4$. Minimum: $W = 2$, $R = 2$ (LOCAL_QUORUM in Cassandra's two-DC terminology). At $W = 2$, every write is confirmed on at least 2 of 3 replicas. At $R = 2$, every read consults at least 2 of 3 replicas. Since $2 + 2 = 4 > 3$, by pigeonhole at least one replica must appear in both the write acknowledgement set and the read response set — the reader always sees a replica that has the write.

c) Engineer A vs Engineer B — which is the better decision for sensor ingestion

Engineer A is making the better decision, for reasons grounded in Section 2.2's stated requirements:

- Consistency requirement: Section 2.2 states 'sensor counters and dashboard aggregates may lag up to 30 seconds' — this explicitly permits eventual consistency.
- Latency requirement: Sensor writes must be acknowledged in <50 ms. QUORUM requires 2 replicas to acknowledge; in a two-DC deployment this adds 5–20 ms per write. Under storm load (120,000 events/s), this pushes p95 write latency above the SLA.
- Availability requirement: 'Sensor ingestion must remain operational during any single-node failure.' ONE consistency means any surviving node can accept writes. QUORUM fails entirely if a partition isolates one node in a 2-of-3 majority requirement.

Engineer B would be correct for billing (where Section 2.2 requires strong consistency). For sensor data, Engineer A's ONE/ONE configuration is the right call.

d) PACELC: Cassandra vs HBase in 24/7 grid monitoring

PACELC (Abadi, 2012) extends CAP with a second dimension. CAP only describes behaviour during a partition (P). PACELC adds: Even during normal operation (E — Else), there is a trade-off between Latency (L) and Consistency (C). This matters enormously for GridSense because network partitions are rare, but latency is a constant operational concern.

Cassandra is classified PA/EL: during a partition it favours Availability; during normal operation it favours Latency. With ONE consistency, write latency is 1–3 ms on the same rack. HBase is classified PC/EC: during partitions and normal operation it favours Consistency. HBase writes go through a single RegionServer leader and must propagate to at least two DataNodes, giving typical write latency of 10–30 ms. Under storm load (120,000 events/s), HBase's leader bottleneck and region split pauses would violate the <50 ms SLA. For a system where partitions are rare but latency spikes have direct operational cost, Cassandra's PA/EL design is the correct choice.

A.3 — Column-Family Data Modelling

a) Table optimised for pattern (1): last N readings for a given sensor

```
CREATE TABLE sensor_readings (  
  sensor_id    TEXT,  
  bucket       TEXT,          -- YYYYMMDDHH (hour-level)  
  reading_time TIMESTAMP,  
  metric_type  TEXT,  
  value        FLOAT,  
  unit         TEXT,  
  quality_flag TINYINT,  
  PRIMARY KEY ((sensor_id, bucket), reading_time)  
) WITH CLUSTERING ORDER BY (reading_time DESC)  
   AND default_time_to_live = 7776000;  -- 90 days
```

The partition key (sensor_id, bucket) means all readings for a given sensor in a given hour are stored contiguously on the same node. A query for 'the last 20 readings from sensor X in the current hour' touches exactly one partition on one node — no cross-node coordination. The DESC clustering order places the most recent reading physically first, so LIMIT 20 reads only the first 20 rows without scanning the rest.

The bucket prevents unbounded partition growth: each partition holds at most 3,600 rows (one per second per hour). Without the bucket, a sensor reporting every second for 90 days would accumulate 7.7 million rows in one partition. With 20 sensors × 24 hours × 90 days = 43,200 distinct partition keys, data distribution across the ring is even.

b) Why pattern (1)'s table cannot serve pattern (2) efficiently

Pattern (2): 'retrieve all sensor readings across the entire network in the last 60 seconds.' The query would require:

```

SELECT * FROM sensor_readings
WHERE bucket = '202605171200'
      AND reading_time >= '2026-05-17 12:59:00'
ALLOW FILTERING;

```

ALLOW FILTERING is required because bucket is part of the composite partition key but sensor_id is not specified — Cassandra cannot know which partitions to scan without the full partition key. The planner broadcasts the query to all nodes and filters results — an $O(\text{total partitions})$ operation, essentially a full-table scan. Even rewritten as 20 separate queries (one per sensor), this requires 20 concurrent round-trips at each dashboard refresh — untenable at 1-second refresh rate for a live network map.

c) Second table for pattern (2) and write amplification analysis

```

CREATE TABLE sensor_readings_by_time (
  time_bucket TEXT,           -- YYYYMMDDHHMM (minute-level)
  reading_time TIMESTAMP,
  sensor_id TEXT,
  metric_type TEXT,
  value FLOAT,
  unit TEXT,
  quality_flag TINYINT,
  PRIMARY KEY ((time_bucket), reading_time, sensor_id)
) WITH CLUSTERING ORDER BY (reading_time DESC, sensor_id ASC)
AND default_time_to_live = 7776000;

```

Dashboard query for pattern (2) now touches a single partition per minute — one partition key lookup, no ALLOW FILTERING. Write amplification: every reading writes to both tables, doubling the total write rate from 40,000 to 80,000 rows/s. In Cassandra terms this is acceptable because both writes use the same LSM-tree append path — no read-before-write, no index rebuild. On a 10-node cluster handling 8,000 events/s per node, the second table adds 8,000 additional events/s per node — manageable with adequate hardware. The explicit trade-off: $2\times$ write I/O in exchange for $O(1)$ reads on both access patterns.

d) Partition key strategy to avoid hot partitions

A partition key of sensor_id alone creates unbounded partitions: one sensor at 1 reading/s for 90 days accumulates 7.7 million rows, causing read amplification and hotspot nodes. The proposed strategy: composite key (sensor_id, bucket) where bucket = YYYYMMDDHH. This guarantees:

- Maximum partition size: 3,600 rows per partition (1 reading/s $\times$ 3,600 s/hour)
- Even distribution: 3.4M sensors $\times$ 24 hour-buckets = 81.6M distinct partition keys/day — distributed evenly across the ring by token hashing
- Range queries preserved: 'last 3 hours for sensor X' requires 3 partition queries with bucket IN (H-2, H-1, H) merged client-side

A.4 — Graph Databases and the Network Topology Problem

a) Why graph databases — SQL vs Cypher concrete comparison

Two-hop downstream traversal in SQL:

```
-- Depth 2
SELECT DISTINCT e2.to_node_id FROM grid_edges e1
JOIN grid_edges e2 ON e1.to_node_id = e2.from_node_id
WHERE e1.from_node_id = 'SS_001';
```

Depth 6 in SQL — requires 5 additional self-JOIN levels:

```
SELECT DISTINCT e6.to_node_id FROM grid_edges e1
JOIN grid_edges e2 ON e1.to_node_id = e2.from_node_id
JOIN grid_edges e3 ON e2.to_node_id = e3.from_node_id
JOIN grid_edges e4 ON e3.to_node_id = e4.from_node_id
JOIN grid_edges e5 ON e4.to_node_id = e5.from_node_id
JOIN grid_edges e6 ON e5.to_node_id = e6.from_node_id
WHERE e1.from_node_id = 'SS_001';
```

Equivalent Cypher — constant regardless of depth:

```
MATCH (origin:Substation {substation_id: 'SS_001'})
-[:FEEDS|SUPPLIES|CONNECTS_TO*1..6]->(downstream)
RETURN labels(downstream)[0], downstream
```

The SQL JOIN count climbs linearly with depth (1 JOIN at depth 2, 5 JOINS at depth 6). Each JOIN materialises an intermediate result that may be as large as the full edge table. Neo4j's traversal follows edges via pointer-dereference from the node record — $O(\text{nodes visited})$, not $O(\text{table size} \times \text{depth})$. The Cypher pattern stays as a single line regardless of depth.

b) Property graph model for GridSense

Label	Key Properties	Notes
GridSupplyPoint	gsp_id, name, voltage_kV, region	Top of supply hierarchy
Substation	substation_id, name, voltage_kV, lat, lon, commissioned_year	Geolocation for field teams
Transformer	asset_id, rating_kVA, manufacturer, model, installed, last_inspection	Links to MongoDB catalog
SmartMeter	meter_id, premise_id, tariff_class, phase	premise_id foreign key to PostgreSQL

Relationship	From → To	Properties (on the edge, not nodes)
FEEDS	GSP → Substation	feeder_id, voltage_kV, length_km, capacity_MW
SUPPLIES	Substation → Transformer	cable_id, distance_m, conductor_mm2
CONNECTS_TO	Transformer → SmartMeter	(logical connection only)

Properties belong on relationships rather than nodes when they describe the connection, not either endpoint. A 132 kV feeder cable of 4.2 km length describes the FEEDS relationship between a specific

GSP and a specific substation. If that substation were connected to a second GSP by a different cable, each connection would have different physical properties. Storing `length_km` on the Substation node would be wrong — it belongs to the connection, not the destination.

c) Neo4j as CP — implications during leader election

In Neo4j's Causal Cluster configuration, writes go to the elected leader via Raft consensus. During a leader election (triggered by leader failure or network partition), the cluster enters an unavailability window of typically 1–10 seconds. Write operations will fail with `ServiceUnavailable` during this window.

Mitigation design: separate the alerting path from the topology write path. Fault alerts go to Redis first (always available as an AP system); topology state updates to Neo4j are done asynchronously via a Redis write buffer. A background worker drains this queue and writes to Neo4j — during a 10-second election, at most a few updates are buffered, and once the new leader is elected, the worker replays them. This guarantees no fault alert is missed while Neo4j momentarily refuses writes.

d) Graph databases for time-series data — critical evaluation

The proposal: store every sensor reading as a node (`:Reading`) connected to its equipment node. Where this remains reasonable: for datasets of 1,000–10,000 readings (e.g., last 24 hours of 10 sensors at 1-minute intervals = 14,400 readings), the graph structure adds genuine value by allowing combined topology+time-series traversal in a single Cypher query.

Where it collapses: GridSense requires 3.4M sensors at 1-second intervals for 90 days:

- $3.4\text{M} \times 86,400 \text{ s} \times 90 \text{ days} = 26.4 \text{ trillion Reading nodes}$
- At ~200 bytes per node (15 bytes node store + relationship + property records): 5.28 petabytes in Neo4j store
- Querying 'what was sensor X reading 3 hours ago' requires traversing a linked list of ~10,800 Reading nodes — $O(N)$ traversal vs Cassandra's $O(1)$ partition seek

Neo4j is not a time-series database. The proposal is interesting for small event sequences (relay events, state transitions) but is operationally impossible at GridSense's sensor volume.

A.5 — Technology Selection Reasoning

Case 1: Complete sequence of relay operations (causal order, immutable, forensic replay)

Technology: Apache Cassandra — `relay_events` table with `TIMEUUID` clustering key

`TIMEUUID` (version-1 UUID) embeds a 60-bit timestamp at 100-nanosecond precision plus a unique node identifier. Unlike a regular `TIMESTAMP`, two `TIMEUUID` values generated at the same millisecond by different relay controllers are still totally ordered — the node ID breaks ties deterministically. This guarantees causal ordering even when events arrive simultaneously during a fault cascade. The `feeder_id` partition key groups all events for a feeder together for forensic replay; ASC clustering order serves them chronologically. Cassandra's append-only write path makes the log immutable once written — there is no `UPDATE` path for a completed SSTable.

Case 2: Find all consumers sharing a transformer with a flagged meter, within two network hops

Technology: Neo4j (Cypher graph traversal)

The access pattern is a two-hop graph query: start at a SmartMeter node, traverse backwards to its Transformer via CONNECTS_TO, then forward to all other SmartMeter nodes connected to that transformer. In Cypher: `MATCH (:SmartMeter {meter_id: $id})<-[:CONNECTS_TO]-(t:Transformer)-[:CONNECTS_TO]->(peer) RETURN peer.meter_id`. This is $O(\text{edges visited})$. In SQL, two self-JOINs on the edge table are required; in MongoDB, two separate queries and client-side joining. The two-hop pattern is the fundamental unit of graph queries — Neo4j executes it natively.

Case 3: Cache overload status of 22,000 transformers with five-second expiry

Technology: Redis (SET with EX)

The pattern is: write 22,000 key-value pairs with a 5-second TTL; read individual transformer status on dashboard requests. Redis SET with EX is a single-command atomic write. With 22,000 keys averaging ~200 bytes each, total memory usage is ~4.4 MB — trivially held in RAM. Read latency is <1 ms per key. Cassandra could store this data but reads are 1–5 ms per key and there is no native per-key TTL-based refresh without a scheduled update job. Redis is the correct tool for ephemeral, frequently-refreshed, low-latency lookup tables.

Case 4: Store a new smart meter with 40 non-standard telemetry fields

Technology: MongoDB (schema-free document storage)

Adding 40 columns to a PostgreSQL equipment table would pollute the schema and make every other row carry 40 NULL columns. MongoDB stores the document natively with only the fields it has — no migration, no schema change, no impact on other documents. The \$set operator with dot-notation (metadata.field_n) enables partial updates that merge new fields without overwriting existing ones. This is the exact use case MongoDB was designed for: heterogeneous documents from different manufacturers with incompatible schemas co-existing in the same collection.

Case 5: Monthly billing for 1.2 million accounts with progressive tariffs — atomically

Technology: PostgreSQL (ACID transactions with JSONB tariff rules)

This is a batch transaction workload requiring ACID guarantees across multiple related tables. Each billing run performs: (1) read tariff_rules JSONB, (2) calculate progressive band charges, (3) insert invoice record, (4) update account balance. These four steps must be atomic — a failure between steps 3 and 4 must roll back the invoice insert. PostgreSQL BEGIN...COMMIT provides this guarantee. Cassandra's lightweight transactions are limited to single-partition operations. MongoDB multi-document transactions add overhead and are less battle-tested for financial workloads. The JSONB tariff_rules column with a GIN index allows querying all accounts with a given tariff plan without schema changes as pricing structures evolve.

Part B — Implementation Overview

The full implementation is at <https://github.com/VaggOpy/gridsense>. A marker can reproduce the running system with two commands:

```
git clone https://github.com/VaggOpy/gridsense.git && cd gridsense
cp .env.example .env
docker compose up --build -d
# wait ~90 seconds for Cassandra to initialise
docker compose exec api python scripts/seed.py
```

B.1 — Services

Container	Technology	Port	Purpose
gridsense_api	FastAPI (Python 3.11)	8000	REST gateway; all business logic
gridsense_cassandra	Apache Cassandra 4.1	9042	Sensor reading time-series storage
gridsense_neo4j	Neo4j 5 Community	7474 / 7687	Network topology & traversal
gridsense_mongo	MongoDB 7	27017	Equipment metadata catalog
gridsense_postgres	PostgreSQL 15	5432	Consumer billing & accounts (ACID)
gridsense_redis	Redis 7 Alpine	6379	Dashboard cache; Pub/Sub fault alerts

B.2 — API Structure

```
api/
├── main.py                # FastAPI app; lifespan DB connections
├── routers/
│   ├── sensors.py         # /sensors - Cassandra
│   ├── grid.py            # /grid - Neo4j
│   ├── equipment.py       # /equipment - MongoDB
│   ├── billing.py         # /billing - PostgreSQL
│   └── alerts.py          # /alerts - Redis Pub/Sub
├── models/               # Pydantic v2 request/response models
├── db/                   # Async database connection factories
├── Dockerfile
└── requirements.txt
```

B.3 — All 14 Mandatory Endpoints (B.7) — Verified

Method	Path	Backend	Status
POST	/sensors/readings	Cassandra	✓ PASS

GET	/sensors/{id}/readings	Cassandra	✓ PASS
GET	/sensors/{id}/summary	Redis cache	✓ PASS
GET	/grid/fault-impact/{node_id}	Neo4j	✓ PASS
GET	/grid/restore-paths/{node_id}	Neo4j	✓ PASS
POST	/grid/nodes	Neo4j	✓ PASS
POST	/grid/relationships	Neo4j	✓ PASS
GET	/equipment/{asset_id}	MongoDB	✓ PASS
POST	/equipment	MongoDB	✓ PASS
PATCH	/equipment/{asset_id}	MongoDB	✓ PASS
GET	/billing/account/{premise_id}	PostgreSQL	✓ PASS
POST	/billing/invoice	PostgreSQL	✓ PASS
GET	/alerts/active	Redis	✓ PASS
POST	/alerts/publish	Redis	✓ PASS

B.4 — Key Design Decisions

Cassandra — Bucketed Partition Keys

Naive design (PRIMARY KEY (sensor_id, reading_time)) creates unbounded partitions. GridSense uses PRIMARY KEY ((sensor_id, bucket), reading_time) where bucket = YYYYMMDDHH. Each partition holds at most 3,600 rows. TTL is set to 90 days (7,776,000 seconds), expiring data automatically without compaction overhead. A second table (sensor_readings_by_time) partitioned by minute-bucket supports cross-sensor dashboard queries — this is the write amplification trade-off described in A.3c.

Neo4j — Cypher Variable-Length Traversal

```
MATCH (origin)-[:FEEDS|SUPPLIES|CONNECTS_TO*1..{max_depth}]->(downstream)
WHERE origin.substation_id = $node_id
RETURN labels(downstream)[0] AS node_type, ...
```

Note: max_depth is interpolated as an f-string integer literal (not a Cypher parameter) because Neo4j 5 does not allow parameters inside variable-length path patterns (*1..\$depth raises SyntaxError). The value is validated as $\text{int} \leq 10$ before interpolation.

MongoDB — Partial Metadata Updates

PATCH /equipment/{asset_id} uses \$set with dot-notation (metadata.key) to merge new fields without overwriting existing ones. This is MongoDB's key advantage for equipment with heterogeneous metadata: a transformer has rating_kVA; a smart meter has communication_protocol; a cable has length_m. Each document carries exactly its own fields.

PostgreSQL — FOR UPDATE Row Lock

The billing invoice endpoint locks the account row with `SELECT ... FOR UPDATE` before reading tariff rules, preventing two concurrent invoice requests from applying different tariff versions to the same account. Row-level locking means requests for different accounts proceed in full parallelism — only genuinely contending requests are serialised.

Redis — Dual-Pattern Alert Delivery

`POST /alerts/publish` performs two Redis operations: `PUBLISH` (for real-time WebSocket subscribers) and `LPUSH + LTRIM` (for REST polling clients). Both happen atomically in the same endpoint — no alert is missed regardless of which pattern a consumer uses. The list is capped at 100 entries with `LTRIM` to prevent unbounded growth. A 24-hour TTL on the list ensures alerts auto-expire without manual cleanup.

B.5 — Data Seeding

Database	Seeded Records	Idempotency Mechanism
Cassandra	50,000 sensor readings across 20 sensors	<code>INSERT</code> (no <code>IF NOT EXISTS</code> needed — same PK overwrites)
Neo4j	2 GSPs, 10 substations, 40 transformers, 200 smart meters + relationships	<code>MERGE</code> (creates only if not exists)
MongoDB	30 equipment records across 3 types (Transformer/SmartMeter/Cable)	<code>insert_one</code> wrapped in <code>try/except DuplicateKeyError</code>
PostgreSQL	100 consumer accounts + 100 invoices	<code>INSERT ... ON CONFLICT DO NOTHING</code>

Part C — Measurement

All measurements were taken on the local development setup: Docker Desktop on Windows, single-node Cassandra cluster, Neo4j Community Edition. The benchmark script is reproducible: `docker compose exec api python scripts/benchmark.py`

C.1 — Cassandra Write Throughput vs Consistency Level

What was measured: end-to-end wall-clock time for writing batches of sensor readings directly through the Cassandra Python driver (bypassing HTTP to isolate DB throughput). Consistency level: ONE — the only meaningful choice on a single-node development cluster (LOCAL_QUORUM and ALL are equivalent to ONE when only one replica exists). Controlled variables: same node, same keyspace, same table, same hour-bucket. Confounds: Docker overlay network adds ~0.5 ms fixed overhead. JVM warm-up in Cassandra is present in the first ~10 iterations; these are included, not discarded.

Batch size	Iterations	Avg (ms)	p50 (ms)	p95 (ms)	Effective writes/s
1	100	0.6	0.6	1.1	~1,544
10	50	5.4	5.2	6.9	~1,862
100	30	51.8	50.2	61.9	~1,929
500	20	262.5	262.6	303.8	~1,904

All batch sizes converge on ~1,900 writes/s. The bottleneck is the driver's synchronous execution model — each `execute()` awaits acknowledgement before the next call. To achieve 40,000 events/s, concurrent writes from multiple threads are required (Cassandra's `execute_concurrent()` API supports this natively). On a 10-node cluster, each node handles 4,000 events/s, scaling linearly.

Why throughput changes between consistency levels on a multi-node cluster: each additional acknowledgement required adds a network round-trip (typically 1–5 ms per hop). With QUORUM on RF = 3, each write awaits 2 replica acknowledgements — ~2 ms additional latency per write, reducing throughput by ~30–40% vs ONE under sustained load.

Production configuration: ONE for sensor ingestion. Section 2.2 permits 30-second staleness for dashboard aggregates; p95 write latency at ONE is 1.1 ms, well within the <50 ms SLA. For a multi-DC deployment, LOCAL_QUORUM provides read-after-write consistency within the local DC at moderate cost — recommended for production if sensor data feeds real-time alerts.

C.2 — Neo4j Traversal Depth vs Latency

What was measured: end-to-end HTTP latency for `GET /grid/fault-impact/SS_001?max_depth={d}` as depth varies from 1 to 8. 35 iterations per depth level (exceeds required 30). Test node: SS_001 — 24 downstream nodes (4 transformers + 20 smart meters) reachable at depth 2; topology exhausted at depth 2 for this node.

max_depth	Nodes returned	Median (ms)	P95 (ms)	Under 200 ms SLA?
1	4	5.2	12.8	YES
2	24	5.7	14.0	YES
3	24	4.7	10.7	YES
4	24	4.8	10.1	YES
5	24	4.9	11.6	YES
6	24	4.6	5.3	YES
7	24	4.9	13.5	YES
8	24	4.8	10.9	YES

The 200 ms SLA is never crossed at any depth level. Latency does not increase with depth: median stays between 4.6 and 5.7 ms across all depths. This confirms $O(\text{nodes visited})$ complexity — once depth 2 is sufficient to reach all reachable nodes, additional depth increments perform no additional work. The traversal terminates when no new edges are found, regardless of the max_depth limit.

Honest extrapolation to 26,000 nodes: these results do not directly generalise. In the seeded topology, only 24 nodes are reachable from SS_001. In a full-scale network, a depth-6 traversal returning 5,000 nodes would take approximately 50–80 ms based on Neo4j's published benchmarks — still within the 200 ms SLA. This should be validated with a full-scale seed before setting production defaults.

C.3 — Redis Cache Effectiveness

What was measured: HTTP latency for GET /sensors/SENSOR_001/summary under cold cache (Redis key deleted before each request) and warm cache (TTL = 30 s, key populated). 50 cold samples, 200 warm samples. Redis key explicitly deleted before each cold measurement.

Condition	p50 (ms)	p95 (ms)	p99 (ms)	Avg (ms)
Cold — Cassandra miss	3.8	4.5	4.6	3.7
Warm — Redis hit	0.8	1.5	1.8	0.9
Speedup (p50)	4.9×			

The warm-cache path (~0.8 ms) is almost entirely Docker inter-container network overhead — Redis's in-memory key lookup is microseconds. The cold-cache path adds ~3 ms for the Cassandra partition read: database query accounts for ~2.9 ms of the cold-cache latency (78% of the differential). In a realistic dashboard scenario (10 engineers × 1 Hz × 30-second TTL), only 1 Cassandra query is needed per 300 requests — a hit rate approaching 99.7%.

allkeys-lru is the correct eviction policy: hot sensors (high-traffic substations) remain in cache while cold sensors (low-activity residential meters) are evicted under memory pressure. allkeys-lfu would produce

marginally better hit rates for this pattern but the practical difference is negligible given the 30-second TTL — keys expire before accumulating meaningful frequency data.

C.4 — Schema Flexibility: MongoDB vs PostgreSQL JSONB

Setup: the same 30 equipment records queried through each database. Mean of 10 runs each.

Query	MongoDB (ms)	PostgreSQL JSONB (ms)	Notes
firmware_version starts with '3.'	3.8	5.1	MongoDB regex index; PG casts JSONB text
SmartMeter with rated_voltage > 230	4.2	6.3	MongoDB numeric comparison; PG casts to numeric
Count grouped by asset_type	2.9	3.7	MongoDB \$group; PG GROUP BY metadata->>'type'

At 30 records, neither difference is operationally significant. The choice of MongoDB for equipment metadata is a schema evolution decision, not a query performance decision. Adding 40 non-standard fields to a MongoDB document requires no migration; adding 40 columns to PostgreSQL requires a migration script and deployment window. At scale (>100,000 records where data exceeds PostgreSQL buffer cache), MongoDB's native document indexing outperforms JSONB for complex nested field queries.

Part D — Reflection

D.1 — The Hardest Decision You Made

The hardest decision was the Cassandra partition key design — specifically whether to include a time bucket in the composite key, and if so, at what granularity.

When I first read the requirements, the starter schema in the assignment used PRIMARY KEY ((sensor_id), reading_time) with no bucket. This is the obvious design: partition by sensor, cluster by time. It is clean, simple, and easy to query. I started with this.

About halfway through implementation I ran into a problem I had not anticipated: partition size. As I seeded 50,000 readings across 20 sensors, some sensors accumulated 2,500 rows per partition. That is small in testing, but the requirement states 3.4 million sensors at up to 1-second intervals for 90 days. A single sensor at 1-second intervals for 90 days would produce 7.7 million rows in one partition. Cassandra's documentation warns against partitions exceeding roughly 100,000 rows for operational stability — query latency grows, compaction becomes expensive, and the node holding that partition becomes a hotspot.

The decision: do I add the bucket and complicate every query, or trust that the marker would not penalise a design matching the starter schema? I genuinely was not sure. The information that would have helped me decide faster: a concrete production data point. I found it eventually in the Discord engineering blog, which described how unbounded partition growth led to GC pauses and eventually required a full migration to ScyllaDB. That was enough. I added the hour-level bucket.

The trade-off I accepted: querying across hour boundaries requires multiple partition queries and client-side merge. That is exactly the write amplification problem described in A.3. What I would do differently: start with the bucket from the first design decision, and use the time saved to add a monitoring endpoint that reports partition size, so the bucket granularity could be tuned in production based on observed sensor reporting rates.

D.2 — Where the Polyglot Architecture Gets Expensive

Burden 1: Cross-system data consistency without distributed transactions.

When an invoice is generated in PostgreSQL, the billing router reads premise_id and assumes a corresponding SmartMeter node exists in Neo4j with the same ID. There is no foreign key constraint between these systems — nothing prevents the Neo4j meter from being deleted while the PostgreSQL account remains active. In production, this creates a class of bugs that are invisible until an engineer queries both systems simultaneously: 'the billing database says this meter exists, but the topology graph says it does not.' Detecting this inconsistency requires a periodic reconciliation job that queries all three systems and compares IDs.

To reduce this burden: designate PostgreSQL as the system of record for account/meter identity and require other systems to sync from it via a changelog (Debezium with Kafka). The cost: a fifth service plus

CDC pipeline operational complexity. Alternatively, accept eventual consistency and build a reconciliation health check into the /health endpoint. Neither is free.

Burden 2: Five different failure modes.

Each database has a distinct failure signature: Cassandra fails with NoHostAvailable when all replicas for a partition are unreachable; Neo4j with ServiceUnavailable during leader elections; MongoDB with connection timeouts under memory pressure; PostgreSQL with connection limit exceeded; Redis with LOADING errors during AOF rewrite. Each requires a different diagnostic approach, different log location, different restart procedure, and different understanding of what data may have been lost.

In this prototype, all five databases are initialized by a single person who built the system. In a production team this means five distinct areas of expertise — either five specialists or five areas where a generalist is inevitably shallow. The operational cost is not visible in the code but becomes very real at 3am when an alert fires and the on-call engineer needs to understand whether the Cassandra node is truly down or doing a major compaction.

The pragmatic mitigation: standardise on a single monitoring interface (Prometheus + Grafana covers all five with the right exporters) and write a runbook for each database. Reducing the five databases to three (merging MongoDB into PostgreSQL JSONB) would reduce the surface area but sacrifice schema flexibility as the equipment inventory grows.

D.3 — Graphs in the Wild

Before this course, I thought about connected data in terms of foreign keys. If sensor SM_00001 belongs to transformer TX_001_A, I would store transformer_id in the sensor table. If I needed to find all sensors downstream of a substation, I would write a JOIN — or if I were clever, a recursive CTE. The mental model was: data lives in tables, relationships are join columns, and traversal is what LEFT JOINS are for.

What changed after working with Neo4j and Cypher is the shift from thinking about data as rows to thinking about it as a network where connections themselves have meaning and properties. In Cypher, the relationship (s:Substation)-[:SUPPLIES {cable_id: 'CB_001', distance_m: 320}]-> (t:Transformer) is not a foreign key or a join table — it is a first-class object carrying its own attributes. That cable has a physical length and a specific identifier, and it belongs on the edge, not on either node. This sounds like a subtle distinction, but it changes how you model problems: instead of asking 'which table should this column go in?', you ask 'does this property describe a node or a connection between nodes?'

A concrete problem in electrical and computer engineering where a graph database would be better than what is typically used: network dependency mapping for embedded firmware in multi-chip systems. A modern SoC design includes hundreds of IP blocks where each block's firmware initialization depends on other blocks being initialised first. This dependency graph — with properties on each edge describing the dependency type (power-on prerequisite, clock source, shared memory region) — is currently stored in spreadsheets or flat XML files in most EDA toolchains.

Finding 'if subsystem X fails to initialise, which other subsystems will fail' is exactly the downstream reachability query that Neo4j handles in O(nodes reached). The current approach — parsing XML and simulating the dependency chain in Python scripts — is fragile, hard to query interactively, and breaks

when an engineer adds a new IP block without updating the dependency matrix. A Neo4j graph of the dependency relationships would enable interactive fault propagation queries, automatic cycle detection (circular dependencies), and visual exploration of the boot sequence — all currently handled with one-off scripts or not at all.

Bibliography

- [1] Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media. [Chapters 5, 6, and 11]
- [2] Gilbert, S. & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59.
- [3] Abadi, D. J. (2012). Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. *IEEE Computer*, 45(2), 37–42.
- [4] Chang, F. et al. (2006). Bigtable: A distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)*, 26(2), 1–26.
- [5] Robinson, I., Webber, J. & Eifrem, E. (2015). Graph Databases: New Opportunities for Connected Data (2nd ed.). O'Reilly Media. [Chapters 2 and 4]
- [6] Vogels, W. (2009). Eventually consistent. *Communications of the ACM*, 52(1), 40–44.
- [7] Lakshman, A. & Malik, P. (2010). Cassandra: A decentralized structured storage system. *ACM SIGOPS Operating Systems Review*, 44(2), 35–40.
- [8] Sadalage, P. J. & Fowler, M. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley.
- [9] Discord Engineering Blog (2023). Migrating Discord's Trillions of Messages from Cassandra to ScyllaDB. engineering.discord.com.
- [10] Neo4j, Inc. (2025). Neo4j Developer Documentation — Cypher Reference. neo4j.com/docs.
- [11] Apache Cassandra Project (2025). Cassandra 4.1 Documentation. cassandra.apache.org.
- [12] Week 1–7 Lecture Slides, Advanced Data Management, ECE — University of Thessaly, Spring 2026.

Appendix A — AI Use Disclosure

In accordance with the assignment's academic integrity guidelines (Section 1.5), this appendix discloses the use of AI assistance in this project.

Tool Used

Claude (Anthropic) — AI assistant used for guidance, debugging, and code generation.

What AI Assisted With

- Boilerplate code: Docker Compose service definitions, FastAPI router structure, Pydantic model definitions, asyncpg/motor/neo4j driver connection patterns
- Debugging: Resolving runtime errors including Cassandra DC name mismatch (DCAwareRoundRobinPolicy → RoundRobinPolicy), Neo4j Cypher SyntaxError (parameter in variable-length pattern), motor/pymongo version incompatibility, and Docker health check timing issues
- Database schema: CQL table definitions, PostgreSQL init.sql, Neo4j seed.cypher
- Documentation structure: README.md skeleton, docs/ file organisation

Student's Work

- The architectural decisions and trade-off reasoning in Part A — the specific positions taken, the calculations presented, and the engineering judgments made (e.g. why ONE consistency is better than QUORUM for sensor ingestion given the stated SLAs) reflect the student's understanding developed through the course
- The reflection essays in Part D — the partition key decision narrative, the specific operational burdens identified, and the SoC firmware dependency graph example are the student's own analysis
- The measurement design and interpretation in Part C — what to measure, how many iterations to use, and what the results mean in context of the SLAs

Prompts Given to the AI

The student directed the AI to: build the project step by step following the assignment PDF, stop at each step for review, explain each decision, and fix errors as they arose during testing. Claude generated code that was tested by the student against the actual running databases and iteratively corrected when it failed.